

Example 5-18. Compute fuzzyness of pandas with accumulators

```
def computeTotalFuzzyNess(sc: SparkContext, rdd: RDD[RawPanda]):  
    (RDD[(String, Long)], Double) = {  
        // Create a named accumulator for doubles  
        val acc = sc.doubleAccumulator("fuzzyNess")  
        val transformed = rdd.map{x => acc.add(x.attributes(0));  
(x.zip, x.id)}  
        // accumulator still has zero value  
        // Note: This example is dangerous since the transformation  
may be  
        // evaluated multiple times.  
        transformed.count() // force evaluation  
        (transformed, acc.value)  
    }
```

Additionally, accumulators support a wide variety of data types provided the operation is associative, but some are easier to get in trouble with than others. To use an accumulator of a different type, you need to implement the `AccumulatorV2[InputType, ValueType]` interface and provide `reset`, `copy`, `isZero`, `value`, `merge`, and `add` methods. You are responsible for the specifics of the class that keeps track of the accumulated values. In general, a simple `var` or two will do the trick. In addition to the required method, override `resetAndCopy` to improve performance in certain cases.

Generally the `reset` and `copy` methods are used together with the `resetAndCopy` method, which can often be more efficiently implemented to avoid the copy stage (as is done in both of the custom accumulator examples, Examples [5-19](#) and [5-20](#)). The `reset` method resets the value of the current accumulator back to “zero” so that `isZero`, if called, will return true. The `copy` method needs to create a copy of the provided accumulator, with the new accumulator having the same value as the current accumulator. This is called when copying the value to the workers so that Spark can avoid the expense (and confusion) of copying any of the previously accumulated work to the drivers.

The type parameters of the `AccumulatorV2` interface specify the type being accumulated over (`add`) and the final return type (`value`). Importantly, this does *not* constrain or specify the type used to hold the accumulation itself. A single variable is used to keep track of values in the following examples. However, you need not limit yourself to one variable. Inside of many of Spark's numeric accumulators, two `vars` are used.

The `merge` method for the accumulator API's type signature takes the same base `AccumulatorV2` type. Since the `AccumulatorV2` trait doesn't specify anything about how workers should keep track of the values as they are evaluated, you will need to cast the accumulator you receive to the expected type so you can access your own internal accumulation field(s). A basic implementation of this is shown in [Example 5-19](#).

Example 5-19. Compute maximum panda id

```
def computeMaxFuzzyNess(sc: SparkContext, rdd: RDD[RawPanda]):  
  (RDD[(String, Long)], Option[Double]) = {  
    class MaxDoubleAccumulator extends AccumulatorV2[Double,  
Option[Double]] {  
      // Here is the var we will accumulate our value in to.  
      var currentVal: Option[Double] = None  
      override def isZero = currentVal.isEmpty  
  
      // Reset the current accumulator to zero - used when sending  
over the wire  
      // to the workers.  
      override def reset() = {  
        currentVal = None  
      }  
  
      // Copy the current accumulator - this is only really used  
in context of  
      // copy and reset - but since it's part of the public API  
let's be safe.  
      def copy() = {  
        val newCopy = new MaxDoubleAccumulator()  
        newCopy.currentVal = currentVal  
        newCopy
```

```

    }

    // We override copy and reset for "speed" - no need to copy
    the value if
    // we are going to zero it right away. This doesn't make
    much difference
    // for Option[Double] but for something like Array[X] could
    be huge.

    override def copyAndReset() = {
        new MaxDoubleAccumulator()
    }

    // Add a new value (called on the worker side)
    override def add(value: Double) = {
        currentVal = Some(
            // If the value is present compare it to the new value -
            otherwise
            // just store the new value as the current max.
            currentVal.map(acc => Math.max(acc,
            value)).getOrElse(value))
    }

    override def merge(other: AccumulatorV2[Double,
    Option[Double]]) = {
        other match {
            case otherFuzzy: MaxDoubleAccumulator =>
                // If the other accumulator has the option set merge
                it in with
                // the standard add procedure. If the other
                accumulator isn't set
                // do nothing.
                otherFuzzy.currentVal.foreach(value => add(value))
            case _ =>
                // This should never happen, Spark will only call
                merge with
                // the correct type - but that won't stop someone else
                from calling
                // merge so throw an exception just in case.
                throw new Exception("Unexpected merge with unsupported
                type" + other)
        }
    }
    // Return the accumulated value.
    override def value = currentVal
}

// Create a new custom accumulator
val acc = new MaxDoubleAccumulator()
sc.register(acc)
val transformed = rdd.map{x => acc.add(x.attributes(0));

```

```

(x.zip, x.id)}
    // accumulator still has None value.
    // Note: This example is dangerous since the transformation
may be
    // evaluated multiple times.
    transformed.count() // force evaluation
    (transformed, acc.value)
}

```

This still requires that the result is the same as the type we are accumulating. If we wanted to collect all of the distinct elements, we would likely want to collect a set and the types would be different. This is shown in [Example 5-20](#).

Example 5-20. Compute unique panda ids

```

def uniquePandas(sc: SparkContext, rdd: RDD[RawPanda]):
HashSet[Long] = {
    class UniqParam extends AccumulatorV2[Long, HashSet[Long]] {
        var accValue: HashSet[Long] = new HashSet[Long]()

        def value = accValue

        override def copy() = {
            val newCopy = new UniqParam()
            newCopy.accValue = accValue.clone
            newCopy
        }
        override def reset() = {
            this.accValue = new HashSet[Long]()
        }
        override def isZero() = {
            accValue.isEmpty
        }

        // We override copy and reset for speed - no need to copy
the value if
        // we care going to zero it right away.
        override def copyAndReset() = {
            new UniqParam()
        }
        // For adding new values
        override def add(value: Long) = {
            accValue += value
        }
        // For merging accumulators

```

```

    override def merge(other: AccumulatorV2[Long,
HashSet[Long]]) = {
      other match {
        case otherUniq: UniqParam =>
          accValue = accValue ++ otherUniq.accValue
        case _ =>
          throw new Exception("only support merging with same
type")
      }
    }
  }
}
// Create an accumulator for keeping track of unique values
val acc = new UniqParam()
// Register with a name
sc.register(acc, "Unique values")
val transformed = rdd.map{x => acc.add(x.id); (x.zip, x.id)}
// accumulator still has Double.MinValue
transformed.count() // force evaluation
acc.value
}

```

TIP

The `value` function can perform complex work and return a different type than the input type or internal accumulated value. For example, if you were computing the average, you might have a `value` function that divides two longs returning a double.

NOTE

You may provide a name for accumulators in Scala so they show up in the web UI. Simply add a name as the second param. This does involve calling `toString` on the accumulator, though—so if that is an expensive operation, leave your accumulator unnamed.

When working with cached data our accumulators can seem almost consistent, but as discussed in [“Interaction with Accumulators”](#) this is not the case.

TIP

There is a proposal to add data property (or “consistent”) accumulators in Spark 2.1.² Property accumulators would avoid double counting—but this remains unmerged. You can follow its progress [in this pull request](#).

Internally, beginning in Spark 2.0, Spark uses accumulators to keep track of task metrics.

Reusing RDDs

Spark offers several options for RDD reuse, including persisting, caching, and checkpointing. However, Spark does not perform any of these automatically³ because storing RDD for reuse breaks some pipelining, which can be a waste if the RDD is only used once or if the transformation is inexpensive to recompute. All kinds of persistence (of which caching is one type) and checkpointing have some cost and are unlikely to improve performance for operations that are performed only once. Furthermore, on large datasets the cost of persisting or checkpointing can be so high that recomputing is more desirable. However, for some specific kinds of Spark programs, reusing an RDD can lead to huge performance gains, both in the terms of speed and reducing failures.

Cases for Reuse

In this section we cover some instances when persisting or checkpointing RDDs may foster performance gains. Broadly speaking, the most important cases for reuse are using an RDD many times; performing multiple actions on the same RDD; and for long chains of (or very expensive) transformations.

ITERATIVE COMPUTATIONS

For transformations that use the same parent RDD multiple times, reusing an RDD forces evaluation of that RDD and so can help avoid repeated computations. For example, if you were performing a loop of joins to the same dataset, persisting that dataset could lead to huge performance improvements since it ensures that the partitions of that RDD will be available in-memory to do each join.

In [Example 5-21](#) we are computing the root mean squared error (RMSE) on a number of different RDDs representing predictions from different models. To do this we have to join each RDD of predictions to an RDD of the data in the validation set.

NOTE

In this example we use `persist()`, which persists the RDD in memory. As we will explain in “Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files”, `cache()` is equivalent to `persist()`, which is equivalent to `persist("MEMORY_ONLY")`.

Example 5-21. A function with iterative computations

```
val testSet: Array[RDD[(Double, Int)]] =  
  Array(  
    validationSet.mapValues(_ + 1),  
    validationSet.mapValues(_ + 2),  
    validationSet)  
validationSet.persist() //persist since we are using this RDD  
several times  
val errors = testSet.map( rdd => {  
  rmse(rdd.join(validationSet).values)  
})
```

Without persisting, Spark would have to reload and repartition the training dataset RDD to complete the `join`. However, with persistence, the

training RDD will stay loaded in memory on the executors with each run of the algorithm. We discuss performance considerations with different kinds of joins in detail in [“Core Spark Joins”](#).

Checkpointing, another form of RDD reuse that writes an RDD to external storage, will also break the RDD’s lineage. However, checkpointing will keep the partitions loaded on the executors.

MULTIPLE ACTIONS ON THE SAME RDD

If you do not reuse an RDD, each action called on an RDD will launch its own Spark job with the full lineage of RDD transformations. Persisting and checkpointing breaks the RDD’s lineage, so the same series of transformations preceding the `persist` or `checkpoint` call will be executed only once. Because persisting or checkpointing an RDD lasts for the duration of a Spark application (although it *may* be evicted by subsequent cached/persisted data), an RDD persisted during one Spark job will be available in a subsequent job executed with the same `SparkContext`. For example, suppose that we wanted to collect the first 10% of the records in an RDD. We could use the code in [Example 5-22](#), which calls `sortByKey`, then `count`, then `take`.

Example 5-22. An example of two actions without a persist step

```
val sorted = rddA.sortByKey()  
val count = sorted.count() // sorted Action 1  
val sample: Long = count / 10  
val sampled = sorted.take(sample.toInt) // sorted Action 2
```

The `sortByKey` (and presumably the read operation) needed to create the RDD, `sorted`, will occur twice if we do not store the RDD: once in the job called by `count` and again in the job called by `take`. We can’t test this element of the execution programmatically, but if you were to run

this application and view the web UI you would see that this code launches two jobs and each one includes a sort stage. However, if we add a persist or checkpoint call before the actions (as shown in [Example 5-23](#)), the transformation will only be executed once, since Spark builds a lineage graph from either an RDD's creation or a persisted/checkpointed RDD.

Example 5-23. Two actions with a persist step

```
val sorted = rddA.sortByKey()  
sorted.persist()  
val count = sorted.count() // sorted Action 1  
val sample: Long = count / 10  
val sampled = sorted.take(sample.toInt) // sorted Action 2
```

TIP

Persisted RDDs only survive for the duration of a Spark application. To reuse data between Spark applications, use checkpointing with the same directory.

IF THE COST TO COMPUTE EACH PARTITION IS VERY HIGH

Even if a program does not use the same RDD multiple times, persisting and checkpointing can speed up a routine and reduce the cost of failures by storing intermediary results. Persisting or checkpointing can be particularly useful if the cost of computing one partition is very high because they ensure that the entire expensive operation will not need to be recomputed in the case of downstream failures.

For example, if your program requires a long series of one-to-one transformations, those transformations will all be combined into very computationally intensive tasks. While this is good so long as the tasks succeed and fit in memory, it does mean that if one of the downstream

transformations fails, then the cost to recompute a single partition may be enormous. If all of the narrow transformations together create more GC overhead or memory strain than your cluster's executors can handle, then checkpointing or persisting `off_heap` can be particularly useful. Both persisting `off_heap` and checkpointing allow the RDD to be stored outside of the Spark executor memory, leaving space to compute. These options are also the only way to prevent recomputation if the entire Spark worker fails. Sometimes breaking up a long lineage graph for its own sake can help a job succeed since it means each of the tasks will be smaller.

Narrow transformations are generally faster than wide ones. However, some individual narrow transformations, such as training a model per partition or working with very wide rows, can be expensive. In these cases, reusing an RDD after the expensive computation so it is not recomputed may improve performance.

Deciding if Recompute Is Inexpensive Enough

Although persisting in memory is a flagship feature of Spark, it is not free. It is space intensive to store data in memory and will take time to serialize and deserialize. As we will discuss in [“Dividing the Space Within One Executor”](#), persisting in memory and in-memory computations are both done in the Spark executor JVM. Thus, persisting in memory may take space that could be used for downstream computations or increase the risk of memory failures. Caching with Java-based memory structures (any of Spark's options besides using `off_heap` storage options) will incur a much higher garbage collecting cost than will recomputing.

Persisting to disk or *checkpointing* (writing the RDD to an external filesystem) has the disadvantages of MapReduce, causing expensive write

and read operations. If the RDD is checkpointed or persisted to disk we must factor in not only the disk space used on the cluster to write the RDD, but also the computational cost on the Spark executors of the additional disk I/O. In most cases, checkpointing a large RDD can be used to reduce failures in high-traffic clusters but rarely leads to performance improvements, even if the RDD has to be recomputed due to the high cost of checkpointing.

TIP

Our experience has been that it is easy to underestimate just how expensive storing and reading an RDD is relative to recomputing. We have also found that for relatively simple operations the cost of the read operation needed to load the RDD far outweighs the others, so persisting is most useful when it prevents triggering another read operation or in the case of many iterative computations.

Furthermore, breaking an RDD's lineage by forcing evaluation through persisting or checkpointing prevents transformations with narrow dependencies from being combined into a single task. Consequently, we lose some of the narrow transformations cannot be combined and executed in one task. For instance, persisting or checkpointing between a simple `map` and `filter` step will break pipelining so that the previously intermediate data can be persisted, causing Spark to do two passes through the data rather than just one, since the transformation has to be evaluated in order to materialize the RDD after the `map`. Breaking lineage between narrow transformations is only desirable in the most extreme cases.

The preceding guidelines are good heuristics for when reuse will provide significant benefits. In general, it is worth reusing an RDD rather than recomputing it if the computation is large relative to your cluster and the

rest of your job. The best way to tell if you need to reuse your RDDs is to run a job. If your job runs very slowly, see if persisting the RDDs may help before attempting to rewrite the program since persisting and checkpointing will help reduce the cost of recomputing data in the case of a failure or eliminate it altogether. If a job is failing with GC or out-of-memory errors, checkpointing or persisting `off_heap` may allow the job to complete, particularly if the cluster is noisy. On the other hand, if you were already persisting with the options that use in-memory persistence consider removing the `persist` call or switching to checkpointing or `off_heap` persistence.

TIP

If you are testing some code before putting it into production, consider creating the persistence level with a variable so that you can pass in a persistence level to try as a command-line argument. The function presented in [Example 5-24](#) uses this paradigm; it contains a `storageLevel` argument (which could be `NONE`).

Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files

If you decide that you need to reuse your RDD, Spark provides a multitude of options for how to store the RDD. Thus it is important to understand when to use the various types of persistence. There are three primary operations that you can use to store your RDD: cache, persist, and checkpoint. In general, caching (equivalent to persisting with the in-memory storage) and persisting are most useful to avoid recomputation during one Spark job or to break RDDs with long lineages, since they keep an RDD on the executors during a Spark job. Checkpointing is most useful to prevent failures and a high cost of recomputation by saving